

Spell Checker & Auto-Complete Engine — Project

Problem

Text input systems need two related capabilities: predicting what a user is about to type (auto-complete) and correcting what they've already typed incorrectly (spell check). Both boil down to the same underlying question — *"which dictionary words are 'close' to this input?"* — just with different notions of closeness: shared prefix for auto-complete, edit distance for spell check.

A naive approach (scanning the full word list for every keystroke or every typo) is fine for a few hundred words, but doesn't scale, and doesn't reflect how these features actually get implemented in real systems like IDEs, search bars, or mobile keyboards.

Approach

The core of the system is a **trie** (prefix tree), where each node's children are stored in a hash map (`unordered_map<char, TrieNode*>`) rather than a fixed 26-slot array. This keeps the structure flexible while still giving average $O(1)$ transitions between characters. Each terminal node also stores a frequency count, and a parallel hash map tracks frequencies globally for $O(1)$ lookup outside of a trie walk.

Auto-complete walks to the node representing the typed prefix, then runs a **DFS** from there to collect every complete word beneath it. Because the DFS only touches the subtree under the prefix, it never wastes time on unrelated branches of the dictionary — results are then sorted by frequency so the most common completions surface first.

Spell checking is the more interesting piece. Instead of computing edit distance between the misspelled word and every dictionary entry independently, the search walks the trie once via DFS, carrying a **Levenshtein-distance row** that gets extended by one cell at each node. This is the standard dynamic-programming recurrence for edit distance, just computed incrementally along trie paths instead of between two fixed strings. Any branch whose *best-case* distance already exceeds the allowed threshold is pruned immediately, so the search naturally avoids exploring dictionary regions that share no useful structure with the typo.

Why this design

The main design decision was picking data structures that make the *shape* of the dictionary do the work, rather than relying on brute-force comparison:

- A hash set alone would give $O(1)$ exact lookup but can't answer "what starts with X" without scanning everything.
- A sorted list would help with prefixes but not with fuzzy typo matching.
- The trie plus incremental edit-distance DFS handles both cases with structure-driven pruning instead of raw iteration.

Results

Tested against a small sample dictionary (12 words across three prefix families — "app-", "ban-", "cat-"):

- Auto-complete for "app" correctly returns apple, application, apply, appreciate, ranked by frequency.
- Spell check for the typo "aple" returns apple, apply — both within edit distance 2, ranked by frequency.
- Spell check for "catt" returns cat, cattle.

Sample output is captured in `examples/sample_output.txt` and the dictionary used is in `data/dictionary.txt`.

What this demonstrates

- Tries for structured string storage and prefix operations
- Hash maps for both node-child storage and $O(1)$ auxiliary lookups
- DFS as the traversal backbone for both collection (auto-complete) and pruned search (spell check)
- Dynamic programming (Levenshtein distance) adapted from a two-string comparison into an incremental, tree-walked form

Possible extensions

- Load a real-world dictionary file (100k+ words) to stress-test pruning efficiency
- Wrap the engine in a small REPL or web API for live demoing
- Blend frequency and edit distance into a single ranking score, rather than treating them as separate sort/filter stages